ARM PrimeCell

Vectored Interrupt Controller (PL192)

Integration Manual

PL192 INTM 0000 A

Open Access

ARM PrimeCell Vectored Interrupt Controller (PL192) Integration Manual

Proprietary notice

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell Vectored Interrupt Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A00	06th September, 2002	First draft
A01	17th September, 2002	Second draft
A	1st October, 2002	Final

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	DDI0273A	ARM PrimeCell Vectored interrupt Controller (PL192) Technical Reference Manual
3	PL192 DDES 0000 A	ARM PrimeCell Vectored Interrupt Controller (PL192) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
VIC	Vectored Interrupt Controller
STA	Static Timing Analysis
ISR	Interrupt Service Routine

2 SCOPE

This document describes how the ARM PrimeCell Vectored Interrupt Controller (VIC PL192) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell Vectored Interrupt Controller is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules has been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell Vectored Interrupt Controller (PL192) Technical Reference Manual [Ref.2].

The ARM PrimeCell Vectored Interrupt Controller (PL192) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell Vectored Interrupt Controller block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

4 INTEGRATION

The Vectored Interrupt Controller block is designed to be integrated into an AMBA system via the AHB (Advanced High-Performance Bus) slave port. It is designed to work in systems where the VIC, through its AHB Slave ports, can raise interrupts to the CPU.

The decode logic within the AHB decoder will need to be altered to take into account the address location of the Vectored Interrupt Controller. The base address of the Vectored Interrupt Controller needs to be defined on a system specific basis and the decode logic altered accordingly. If AMBA TIC methodology is to be used for integration testing then any change of address location for the Vectored Interrupt Controller block must also be reflected in the AMBA TIC test vectors, by amending the base address declaration for the Vectored Interrupt Controller integration test program.

4.1 Supported Design Flow

The ARM PrimeCell Vectored Interrupt Controller has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell Vectored Interrupt Controller block.

An example ASIC design flow is described below:

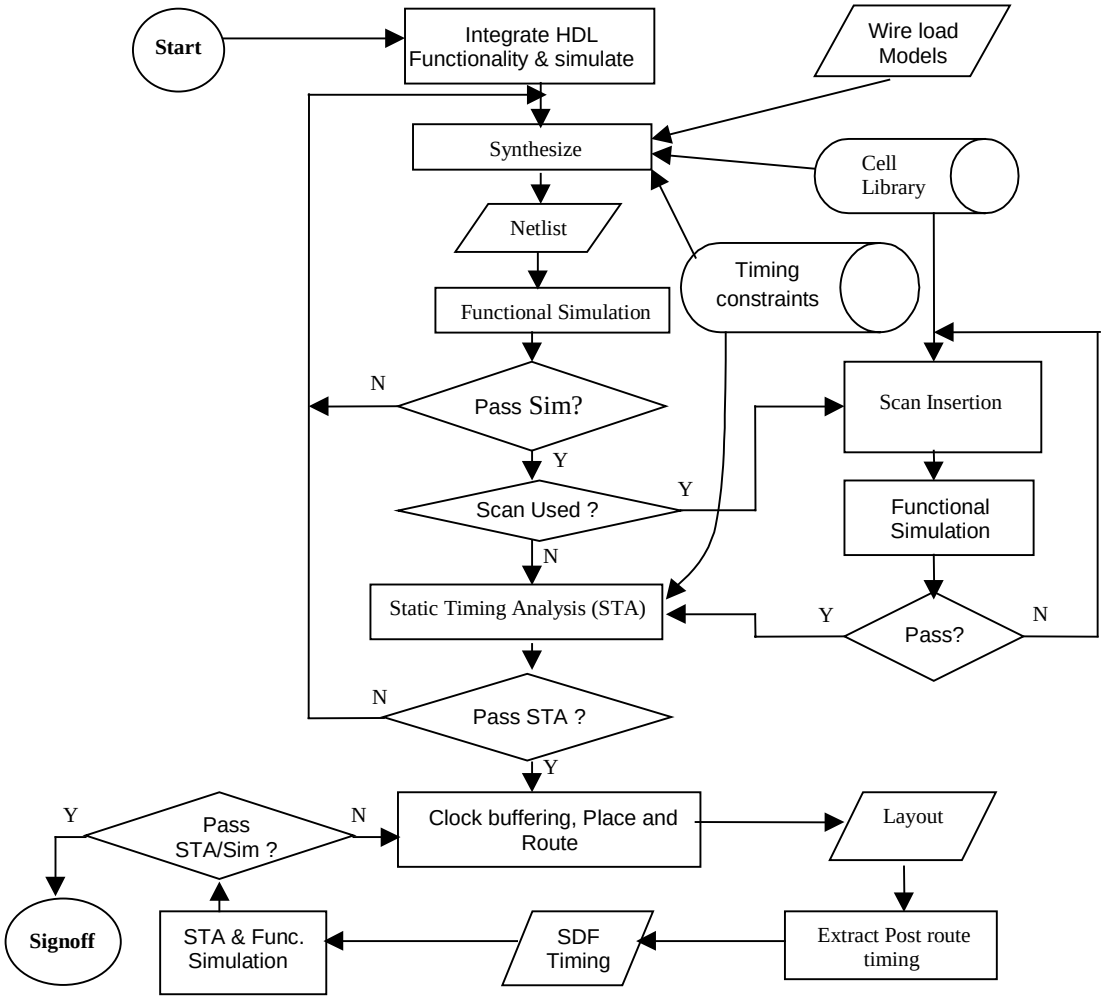

Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Description

The following tables describe the Vectored Interrupt controller interface signals. ***Tables 4.2-1*** lists the AHB slave interface signals. ***Tables 4.2-2*** and ***4.2-3*** list the miscellaneous signals and the scan test related signals in the VIC. For further information refer to the ARM PrimeCell Vectored Interrupt Controller (PL192) Technical Reference Manual [Ref.2].

For information on the AHB, see AMBA Specification Rev2.0 [Ref. 1].

Signal Name	Type	Source / Destination	Description
HCLK Bus clock	Input	Clock Source	This clock times all bus transfers. All signal timings are related to the rising edge of HCLK.
HRESETn Reset	Input	Reset controller	The Bus reset signal is active LOW and is used to reset the system and the bus.
HADDR[11: 2] Address bus	Input	AHB Master	The AHB address bus to the AHB interface.
HWRITE Transfer direction	Input	AHB Master	When HIGH this signal indicates a write transfer and when LOW, a read transfer to the registers.
HSIZE[2:0] Transfer size	Input	AHB Master	Indicates the size of the transfer to the registers. The VIC AHB Slave register Interface supports only 32-bit accesses ‘010’ represents 32-bit. If HSIZE[2:0] indicates a transfer width other than 32-bits, the VIC returns an ERROR response on HRESP[1:0]
HTRANS[1:0] Transfer type	Input	AHB Master	Indicates whether a data-transaction is to be done for the current transfer to register. ‘00’ represents IDLE. ‘01’ represents BUSY. ‘10’ represents NSEQ ‘11’ represents SEQ The registers will be updated only when the HTRANS is either NSEQ or SEQ transfers.
HREADYIN Transfer done	Input	AHB Slave	When HIGH the HREADYIN signal indicates that a transfer has finished on the bus. When LOW, it indicates that the AHB Slave has introduced a wait state.
HSELVIC VIC Slave select	Input	AHB Decoder	When HIGH, HSELVIC indicates that the current transfer is intended for the VIC. This signal is a combinatorial decode of the address bus.
HWDATA[31:0] Write data bus	Input	AHB Master	The write data bus is used to transfer data from the master to the VIC AHB Slave during write operations. The VIC AHB Slave supports only 32-bit accesses.
HRDATA[31:0] Read data bus	Output	AHB Master	The read data bus is used to read data from the VIC AHB Slave Interface. The VIC AHB Slave supports only 32-bit accesses.

Signal Name	Type	Source / Destination	Description
HREADYOUT Transfer done	Output	AHB Master	When HIGH the HREADYOUT signal indicates that a transfer targeting the VIC AHB Slave has finished on the bus. When LOW, it indicates that a wait state has been introduced by the AHB Slave
HRESP[1:0] Transfer response	Output	AHB Slave	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. The VIC AHB Slave only drives OKAY or ERROR response.

Table 4.2-1 AHB Slave Interface signals

Signal Name	Type	Source/ Destination	Description
VICINTSOURCE[31:0] Interrupt source	Input	Peripheral	Peripheral Interrupt source interrupt request
nVICSYNCEN Synchronisation signal to the vic port	Input	System	Used to indicate that the VIC port must operate in asynchronous mode when tied HIGH. Synchronous operation used when tied LOW.
VICIRQACK Acknowledgement signal from the CPU	Input	Processor	Interrupt acknowledge from the processor. Used during the processor/VIC handshaking after an IRQ is asserted.
nVICFIQIN FIQ interrupt from the daisy chain VIC	Input	External Interrupt Controller	Connects to the nVICIRQ signal of the previous VIC if daisy chaining is used. Connects to logic ‘1’ if the VIC is the last in the daisy chain or if the VIC is not daisy chained.
nVICIRQIN IRQ interrupt from the daisy chain VIC	Input	External Interrupt Controller	Connects to the nVICFIQ signal of the previous VIC if daisy chaining is used. Connects to logic ‘1’ if the VIC is the last in the daisy chain or if VIC is not a daisy chained.
VICVECTADDRIN[31:0] Address In from the daisy chain VIC.	Input	External Interrupt Controller	Connects to the VICVECTADDROUT[31:0] signal of the previous VIC if daisy chaining is used. Connects to logic ‘0’ if the VIC is not daisy chained.
VICFIQINREG Register enable signal to nVICFIQIN	Input	System	Tied HIGH if the user wishes to register the daisy chained nVICFIQIN input into the VIC. Tied LOW if the user does not wish to register the nVICFIQIN into the VIC.
VICIRQINREG Register enable signal to nVICIRQIN	Input	System	Tied HIGH if the user wishes to register the daisy chained nVICIRQIN input into the VIC. Tied LOW if the user does not wish to register the nVICIRQIN into the VIC.

Signal Name	Type	Source/ Destination	Description
nVICFIQ	Output	Processor	Fast Interrupt request to processor
nVICIRQ	Output	Processor	Interrupt request to processor
VICVECTADDROUT[31:0]	Output	Processor	Connects to the VICVECTADDRIN[31:0] signal of the next VIC if daisy chaining is used. Left unconnected if the VIC is not daisy-chained or core does not support VIC port.
VICVECTADDRV	Output	Processor	Indicates that the VICVECTADDROUT bus contains a stable value. Used during the processor/VIC handshaking after an IRQ is generated.
VICIRQACKOUT	Output	External Interrupt Controller	Interrupt acknowledge from the processor, which has passed through the VICs in the daisy chain. Used during the processor/VIC handshaking after an IRQ is generated.

Table 4.2-2 Non AMBA signals

Signal Name	Type	Source / Destination	Description
SCANENABLE	Input	Test Controller	Scan Enable signal. Indicates that the circuit is in scan-test mode.
SCANINHCLK	Input	Test Controller	HCLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTHCLK	Output	Test Controller	HCLK domain Scan data output for clocking in the test pattern in scan-test mode.

Table 4.2-3 Scan Test related signals

4.3 AMBA bus connection

The AHB register interface must be connected to the AHB bus with the microprocessor so that the microprocessor can program the VIC.

The VIC block interfaces with the AMBA AHB interfaces using separate unidirectional read and write data buses HRDATA and HWDATA. For further information refer to the AMBA AHB Specification [Ref.1].

The AHB data buses from the VIC should be interfaced with other data buses from other peripherals using multiplexers at the chip level.

4.4 Non-AMBA Intra-chip connection

The non-AMBA signals connections are divided into three different types as mentioned below,

- Single interrupt controller connectivity to a processor without VIC port

- Daisy chained interrupt controller connectivity without VIC port
- VIC port connection

Single interrupt controller connectivity to a processor without VIC port

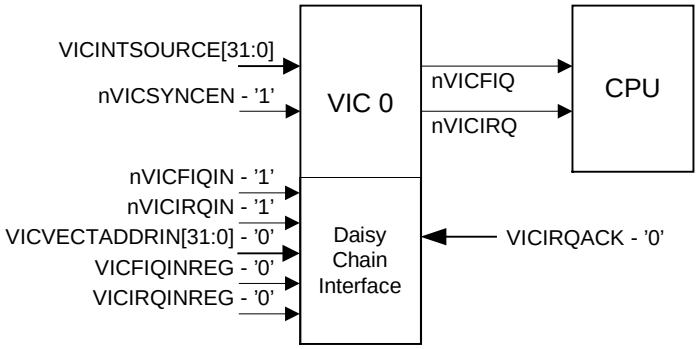

Figure 4.4-1: Single Interrupt controller with out VIC port connection

The description on signal connection is as described below,

- The interrupt requests from the peripherals are connected to the VICINTSOURCE pins. These are the hardware interrupts.
- Connect the nVICFIQIN and nVICIRQIN to logic ‘1’.
- VICVECTADDRIN[31:0] is connected to logic ‘0’.
- VICFIQINREG and VICIRQINREG are connected to ‘0’.
- nVICSYNCEN is connected to ‘1’. VICIRQACK is connected to logic ’0’.
- The pins nVICIRQ and nVICFIQ are connected to the nIRQ and nFIQ pins of the processor.

Daisy-chained interrupt controller connectivity without VIC port

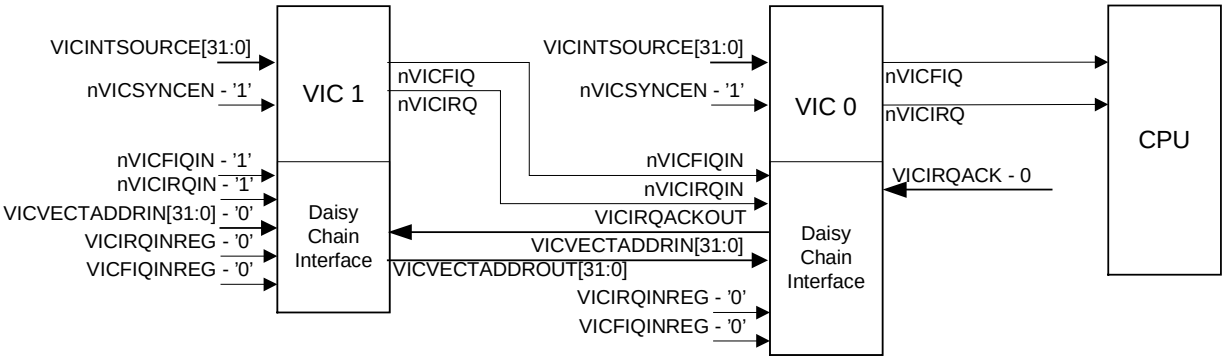

Figure 4.4-2: Daisy-chained interrupt controller connectivity without VIC port

The description of the signal connections are described below when VIC1 and VIC0 (nearer to processor) are connected in daisy-chain fashion,

- nVICIRQIN of VIC0 is connected to the nVICIRQ output of VIC1. nVICFIQIN on the VIC0 connected to the nVICFIQ output of VIC1.
- VICVECTADDRIN[31:0] on the VIC0 connected to the VICVECTADDROUT[31:0] of the VIC1.
- VICIRQACK on VIC0 tied to logic ‘0’.

- VICIRQACKOUT on VIC0 connected to VICIRQACK input of VIC1.
- VICIRQINREG and VICFIQINREG on all VICs are tied to logic ‘0’ if it is not required to clock the VICIRQINREG and VICFIQINREG.
- If it is required to register the nVICIRQIN and nVICFIQIN then tie the VICIRQINREG and VICFIQINREG to logic ‘1’.

VIC port connections:

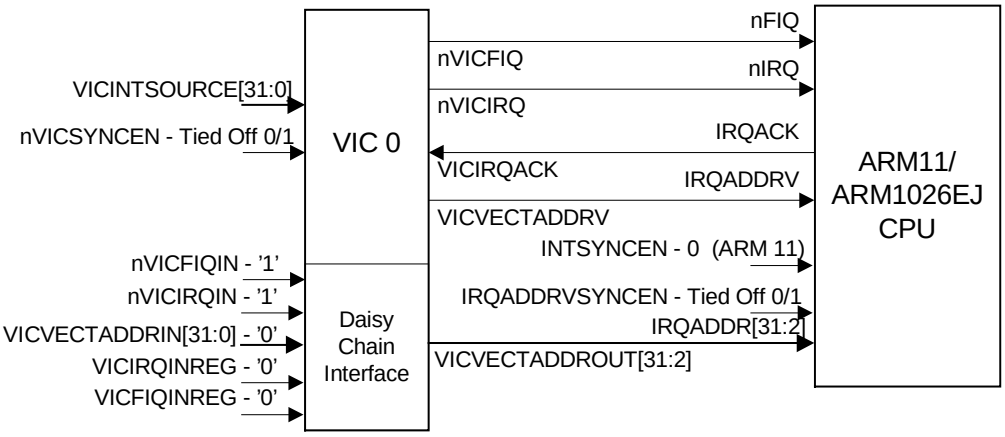

Figure 4.4-3: VIC port connections

- VICIRQACK is connected to the IRQACK pin of the processor and VICVECTADDRV is connected to IRQADDRV pin of the processor.
- VICVECTADDROUT is connected to IRQADDR of the processor.
- The synchronous/asynchronous interface control port IRQADDRVSYNCEN must be tied off to the same value as the nVICSYNCEN to ensure that both the VIC and the processor are operating in the same mode.
- INTSYNCEN is always set to 0 because the PL192 VIC, nVICIRQ and nVICFIQ outputs are asynchronous.

4.5 Clocks

The ARM PrimeCell VIC HDL contains a clock source at the top level – HCLK.

HCLK is the AHB clock used for the AHB side signals and to synchronise the signals.

Please refer to the ARM PrimeCell Vectored Interrupt Controller (PL192) Technical Reference Manual [Ref 2] for more details on the clocks in the VIC.

4.6 Resets

The VIC block has one reset pin, HRESETn. HRESETn is the system bus reset

The system reset controller can assert HRESETn asynchronously, but it should be deasserted synchronous to the rising edge of HCLK. HRESETn is used as the reset for all the flip-flops used in the design.

4.7 Scan Signals

SCANENABLE is the scan-mode enable pin of the VIC. A separate scan input and output is provided. The various scan signals are described in ***Table 4.2-3***.

5 SYNTHESIS

The Vectored Interrupt Controller block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script – Vic.cmd - synthesizes the Vectored Interrupt Controller to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Vic_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

5.1 Constraints

The top level VIC is synthesized for the frequency of 166 MHz.

The constraints on the different inputs and outputs of the VIC are described in the following sub-sections.

5.1.1 AMBA bus signals

The AMBA signals are synthesised at the following constraints:

- 30 % input setup on all inputs to the AHB Slave ports and the AHB Master port
- 40 % output valid delay on all outputs of the AHB Slave ports and the AHB Master port

5.1.2 Non-AMBA Intra-chip signals with direct input to output connection

The different Intra-chip signals are synthesised at the following constraints:

- 80% input setup on VICINTSOURCE, nVICFIQIN, nVICIRQIN, VICVECTADDRIN, VICFIQINREG, VICIRQINREG, nVICSYNCEN inputs.
- 25% input setup on VICIRQACK input.
- 85% output valid delay on VICIRQACKOUT output.
- 55% output valid delay on nVICIRQ, nVICFIQ output.
- 50% output valid delay on VICVECTADDROUT output.
- 40% output valid delay on VICVECTADDRV output.

5.1.3 Clocks

Many different design methodologies are available for clock tree insertion and synthesis. The default Vectored Interrupt Controller synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that a separate tool, such as place & route tools, which understand placement, further on in the backend process, will perform clock buffering. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

5.1.4 Resets

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block

5.1.5 Scan signals

False paths are not set to any of the paths.

5.2 Synthesis scripts

The synthesis scripts consist of the following three main categories:

5.2.1 Setup Scripts

Scripts that are common to several designs are located in the $GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- Target area goals
- Maximum transition costs

The library_avanti_cb18_v1.0.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb18_v1.0.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The $LIB_SYNOP directory must contain the files or links to the target cell library files, in Synopsys format.

5.2.2 Command Script

The master compile command script Vic.cmd in the synopsys/scripts directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The Vic.cmd script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script Vic_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- $PERIPH is assigned to ‘topname’ to define the name of the top-level module in the design.

- $HDL_SOURCE is assigned to ‘hdl’ to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- $HDL_NETL is assigned to ‘hdl_out’ to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

5.2.3 Constraint Scripts

a) Operating conditions and signal load/drive values should be set for the chosen technology library in a file equivalent to the library_avanti_cb18_v1.0.scr file.

b) Timing constraints for AHB bus signals are set via the amba_params.scr, ahb_master.scr and the ahb_slave.scr files. As delivered, the timing for the VIC is set for a 166 MHz clock. The timings should be adjusted according to the target frequency required. The above-mentioned files have been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

   Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

   - The minus skew is set at 2.5% of the clock period. This is subtracted from the clock period used for the synthesis
   - The plus skew is set at 40% of minus skew. This is used for the plus uncertainty value.

   The plus skew derives from the best case condition and it is less than the minus skew, which derives from worst case condition.

c) Timing constraints for non-AMBA VIC-specific signals are set via the Vic.scr file for the VIC. Sample input and output delay constraints are specified for these signals. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

d) Timing exceptions are specified via the Vic_exceptions.scr file. In this design there is no false path set on any signal.

6 INTEGRATION TEST

The Vectored Interrupt Controller integration test vector suite allows the integrator to verify that the Vectored Interrupt Controller has been connected into the target system correctly.

6.1 Integration Test strategy

Integration tests are required to test three classes of signals:

6.1.1 AMBA signals

The AHB signals connected to the AHB Slave ports of the Vectored Interrupt Controller signals are tested with AHB transactions to the VIC. The vectors required for testing the AHB register port are present in the RegAccessTests.c file in the integration/bustest directory.

6.1.2 Non-AMBA intra-chip signals

Running the Integration test vectors present in the IntegrationTest.c file in the integration/bustest directory tests the non-AMBA intra-chip signals of the VIC.

6.2 Integration Test Code

The integration/bustest directory contains the integration test code files, IntegrationTest.c and RegAccessTests.c. The file Vic.c in the integration/bustest directory is used to selectively run the integration tests or all of these tests. As delivered, the integration test vectors are intended to run correctly with the Vectored Interrupt Controller in stand-alone mode within the integration EASY world i.e. even before the Vectored Interrupt Controller has been integrated into the user system.

After integration, the following points need to be noted:

- The integration vectors targeting the Vectored Interrupt Controller AHB slave port (AMBA signals) connectivity do **NOT** have to be modified by the integrator.
- The integration test vectors targeting the intra-chip signals of the VIC are required to be modified to drive a ‘1’ or a ‘0’ on the intra-chip inputs and to read the intra-chip outputs. The test vectors that need to be modified have been indicated in the IntegrationTest.c file in the /integration/bustest directory.

7 PRODUCTION TEST

The Vectored Interrupt Controller is designed to be production tested using scan-insertion and ATPG.

7.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The Vectored Interrupt Controller design fully supports this scan test and ATPG methodology.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The system designer does extraction of test vectors once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

8 FUNCTIONAL AND TIMING VERIFICATION

It is essential to verify the functionality and the timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC

8.1 Functional verification

Functional verification is used to test the functionality of the design. The netlist should also be checked for functional equivalence to the RTL at all the key stages of the design flow. The functional verification can be performed by one of the two methods specified below:

8.1.1 Formal Verification

Formal verification is used to compare RTL with RTL and RTL with netlist. The functionality of the netlist or RTL can be compared to that of the RTL by use of a logic equivalence-checking tool (for example, Synopsys Formality or Chrysalis). Functionality of the netlist and RTL can be compared within a shorter period of time by this method when compared to gate-level simulations.

8.1.2 Dynamic simulation

Verification tests are run on the RTL and netlist to confirm functionality of the design. Gate-level simulations can be used to compare the netlist functionality to that of the RTL. The same test vectors that were used to verify the RTL are used to verify the netlist functionality. This is however a time consuming process.

8.2 Timing verification

It is essential to verify timing performance of the netlist against the RTL at all the key stages of the design flow. Use of timing verification tools is a useful check for human error in the manual definition of false paths. The following two methods can be used to perform timing verification:

8.2.1 Static Timing analysis

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis.

8.2.2 Dynamic simulation

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

9 CUSTOMISATION

The ARM PrimeCell Vectored Interrupt Controller can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the test benches where required. Synthesis will have to be rerun to ensure that timing is met. Information to support such modification is included in the ARM PrimeCell Vectored Interrupt Controller (PL192) Design Manual [Ref.3].

The following sections describe some of the customisations that are possible in the VIC and estimates of the gate count reduction / addition, the time required and difficulty level of the change and the RTL updates required for each customisation. The gate counts that will be achieved are dependent on the target cell library used, and synthesis timing constraints and command options.

9.1 Removing the VIC port

The ARM PrimeCell VIC (PL192) supports the VIC port. It is possible to remove the VIC port support so that only software handshaking is supported.

9.1.1 Gate Count change

Removal of VIC port reduces the gate count by around 1K.

9.1.2 RTL updates required

Remove the following pins

VICIRQACK,

nVICSYNCEN,

VICVECTADDRV

VicCpuif block:

- Remove IRQACKtmux.
- Remove the multiplexer, which selects the VICIRQACK depending on the value of nVICSYNCEN.
- Generation of VICIRQACKOUT should be modified so that it is devoid of VICIRQACK.
- Remove the logic, which generates VADDRVtmux, which is used as VICVECTADDRV.
- Remove the logic for the generation of ReadyForAck. Generation of StackPush depends on the value of VICIRQACK (LastIrqAck). Modify it in such a way that it only depends on software acknowledgement.
- Modify the CurrentPriority generation in such a way that the CurrentPriority always follows the value of PriorityStack value.
- Remove the integration register fields, which are related to VIC port.

9.1.3 Time required for the change and difficulty level:

Approximate estimate: Between 1 and 2 person days, assuming similar experience.

Difficulty level: Medium.

9.2 Removing the Daisy Chain Connection support

The ARM PrimeCell VIC (PL192) supports the daisy chain connection support where more than one VIC can be connected in Daisy Chain connection fashion.

9.2.1 Gate Count change

Removal of Daisy chain support can reduce the gate count by 2K.

9.2.2 RTL updates required

Remove the following pins

nVICFIQIN,

nVICIRQIN,

VICVECTADDRIN[31:0],

VICVECTADDROUT[31:0],

VICIRQACKOUT,

VICIRQINREG,

VICFIQINREG.

VicIntResolver block:

- Remove the generation of DaisyChainIn.
- Modify the generation of nVICFIQ.
- Remove the integration register fields, which are concerned with daisy chain interface.

VicPriority block:

- Reduce the size of LevelMatchedIrq[32:0] and SyncLvMatchedIrq[32:0] to LevelMatchedIrq[31:0] and SyncLvMatchedIrq[31:0].
- Remove the logic, which updates the 32nd bit of above signals. Modify the generation of IRQPort in similar way.

VicCpuif block:

- Remove the logic of generation of VICIRQACKOUT.
- Remove the VICVECTADDRIN from the mux, which selects the valid interrupt address.

9.2.3 Time required for the change and difficulty level:

Approximate estimate: Between 1 and 2 person days, assuming similar experience.

Difficulty level: Medium.

10 APPENDIX A

10.1 PrimeCell Environment Variables

Simulation and Synthesis in the Vectored Interrupt Controller world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the **'GLOBAL'** variable.

e.g.:

- **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the **'PERIPH'** variable.

e.g.:

- **setenv PERIPH Vic**

SIMULATOR

The simulator type is defined by the **'SIMULATOR'** variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

e.g.:

- **setenv SIMULATOR modelsim**
- **setenv SIMULATOR LDV**

TEST_METH

The scan methodology adopted during synthesis is defined by the **'TEST_METH'** variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scaninsert - creates a fully routed scan netlist.
- scanready - creates a netlist without scan routing

e.g.:

- **setenv TEST_METH noscan**
- **setenv TEST_METH scaninsert**
- **setenv TEST_METH scanready**

Note:

All the scan inputs and outputs are specified at the top-level of the VIC.

HDL_SOURCE

The source code type is defined by the **'HDL_SOURCE'** variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

e.g.:

- **setenv HDL_SOURCE vhdl**
- **setenv HDL_SOURCE verilog**

HDL_NETL

The netlist type is defined by the **'HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

e.g.:

- **setenv HDL_NETL vhdl**
- **setenv HDL_NETL verilog**

TEST_ENV

The test environment is defined by the **'TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g.:

- **setenv TEST_ENV BUSTALK**
- **setenv TEST_ENV TICTALK**

LIB_VHDL

The location of the vhdl cell library views is defined by the **'LIB_VHDL**' variable.

e.g.:

- **setenv LIB_VHDL /AVANT/cb18/v1.0/vital/cb18os120/zero**

DIRS_VERILOG

The location of the verilog cell library views is defined by the **'DIRS_VERILOG'** variable.

e.g.:

- **setenv DIRS_VERILOG /AVANT/cb18/v1.0/verilog/cb18os120/zero**

FILES_VERILOG

The location of the verilog UDP (User Defined Primitive) file is defined by the **'FILES_VERILOG'** variable.

e.g.:

- **setenv FILES_VERILOG /AVANT/cb18/v1.0/verilog/cb18os120/zero/mtb_verilog.v**

LIB_SYNOP

The location of the synopsys target library files is defined by the **'LIB_SYNOP'** variable.

e.g.:

- **setenv LIB_SYNOP /AVANT/cb18/v1.0/synopsys/cb18os120/1999.05/models**

AMBA

This environment variable defines the AMBA bus to which the peripheral is connected - AHB, ASB or APB. Since the VIC PL192 is an AHB peripheral, set this environment variable to 'AHB'. This environment variable is used by makefiles in the VIC database to select settings for the Integration world.

e.g.:

- **setenv AMBA AHB**